

Advanced Programming

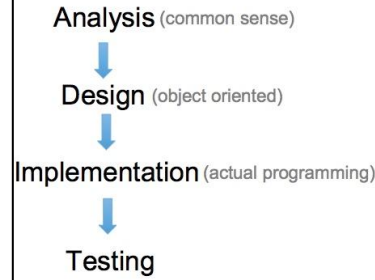
CSE 201

Lecture 17: UML

What is UML?

- UML stands for Unified Modeling Language
- It's a widely used modeling language in the field of software engineering
- It's used to analyze, design, and implement software-based systems
- Pretty pictures (diagrams) →

Ideas to Program



- Analysis
 - **What to do and not how to do it**
 - Decide corner cases and exact functionalities
- Design
 - Define classes, their attributes and methods, objects, and class relationships
- Implementation
 - Novice programmers often think that writing code is the heart of software development, but actually it should be the least creative step
- Testing
 - A program should be free of errors

© Vivek Kumar

4



www.shutterstock.com · 464893271

Motivations for UML

- We need a modeling language to:
 - help develop efficient, effective and correct designs, particularly Object Oriented designs
 - communicate clearly with project stakeholders (concerned parties: developers, customer, etc)
 - give us the “big picture” view of the project

UML Diagrams

Three types of UML diagrams:

1. **Class diagrams:** show the classes in a system, attributes, and operations of each class and the relationship between each class.
2. **Use case diagrams:** give a graphic overview of the actors involved in a system, different functions needed by those actors and how these different functions interact.
3. **Sequence diagrams:** show how objects interact with each other and the order of those interactions.

UML Diagrams: Class Diagrams

A class diagram in the Unified Modeling Language (UML) is a type of static structure diagram that describes the structure of a system by showing the system's:

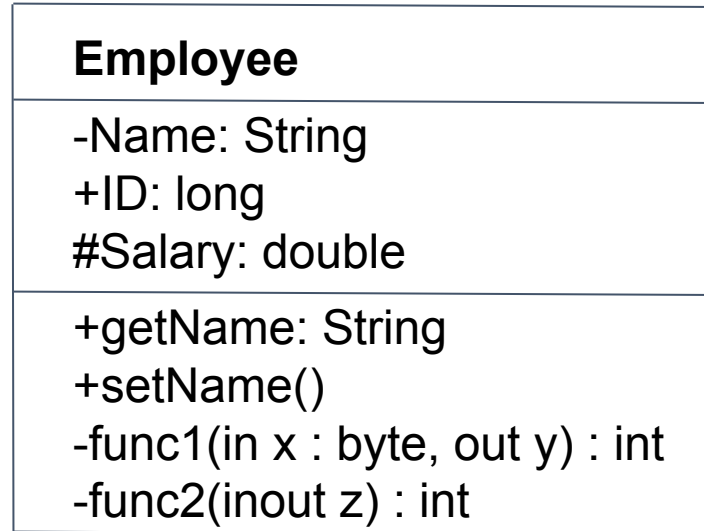
- classes,
- their attributes,
- operations (or methods),
- class relationships.

UML Class Notation

- A class is a rectangle divided into three parts
 - Class name
 - Class attributes
 - Class operations

- Modifiers
 - Private: -
 - Public: +
 - Protected: #
 - Static: Underlined

- Abstract class/methods
 - Name in italics

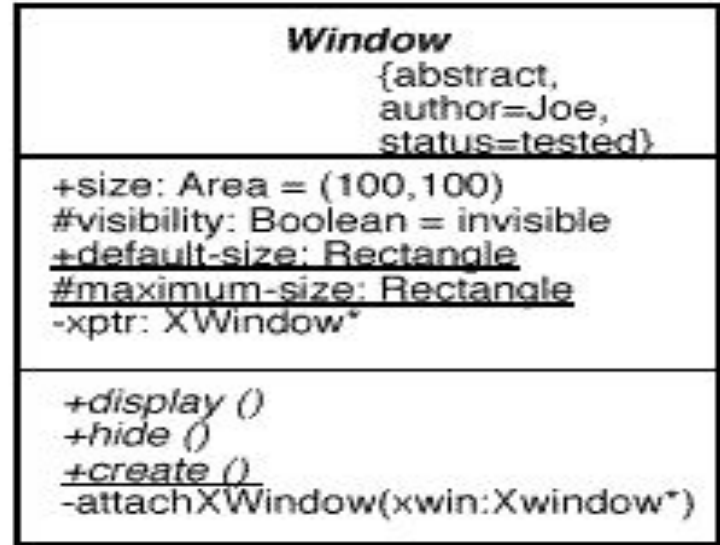
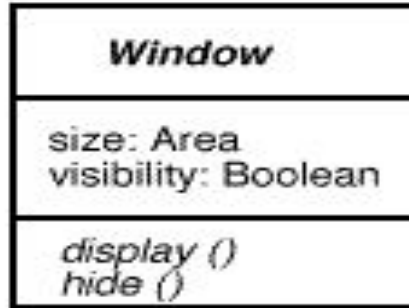


Different Levels of Specifying Classes

Conceptual



Specification

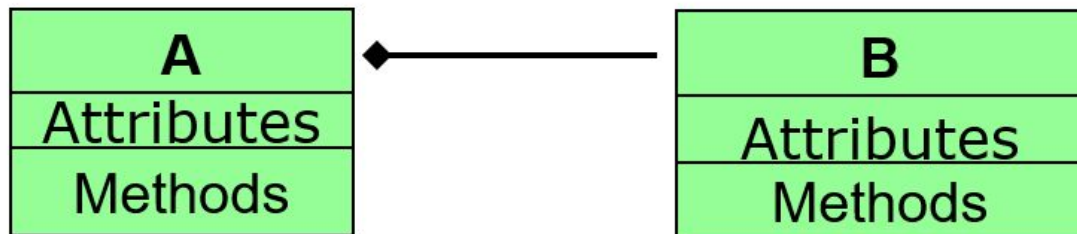


Implementation

Use this for your project

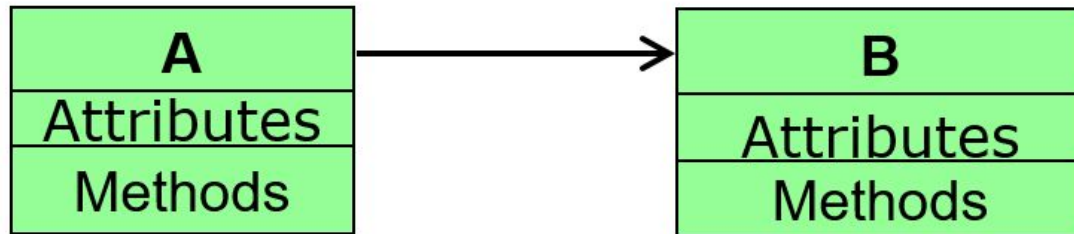
Composition in UML

- Represented by a solid arrow with diamond head
- In below UML diagram, A is composed of B



Association in UML

- Represented by a solid arrow
- In below UML diagram, A holds a reference of B



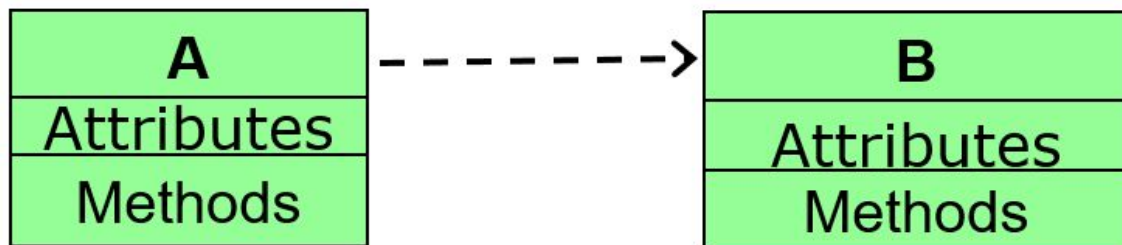
Class Relationship: Binary Association

Both entities “Knows About” each other (two-way association)

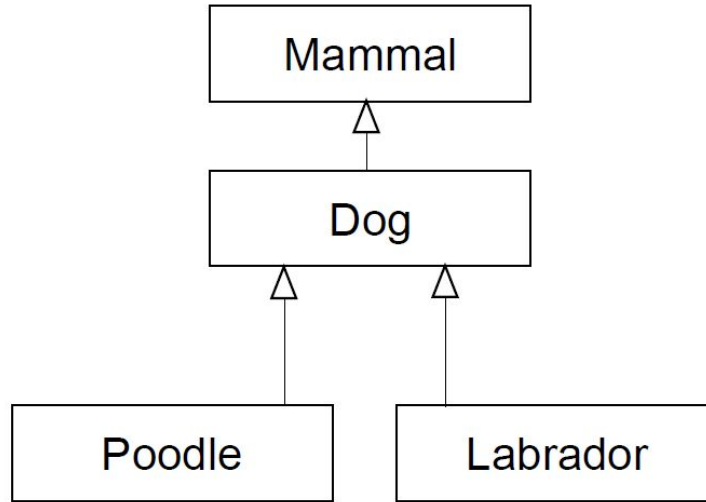


Dependency in UML

- Represented by a dashed arrow starting from the dependent class to its dependency
 - A is dependent on B
 - A is requesting service from B



Inheritance in UML



UML Multiplicities

Links on associations to specify more details about the relationship

Multiplicities	Meaning
0..1	zero or one instance. The notation " <i>n</i> .. <i>M</i> " indicates <i>n</i> to <i>m</i> instances.
0..* or *	no limit on the number of instances (including none).
1	exactly one instance
1..*	at least one instance

Multiplicity example



A customer can have 1 to 5 bank accounts.



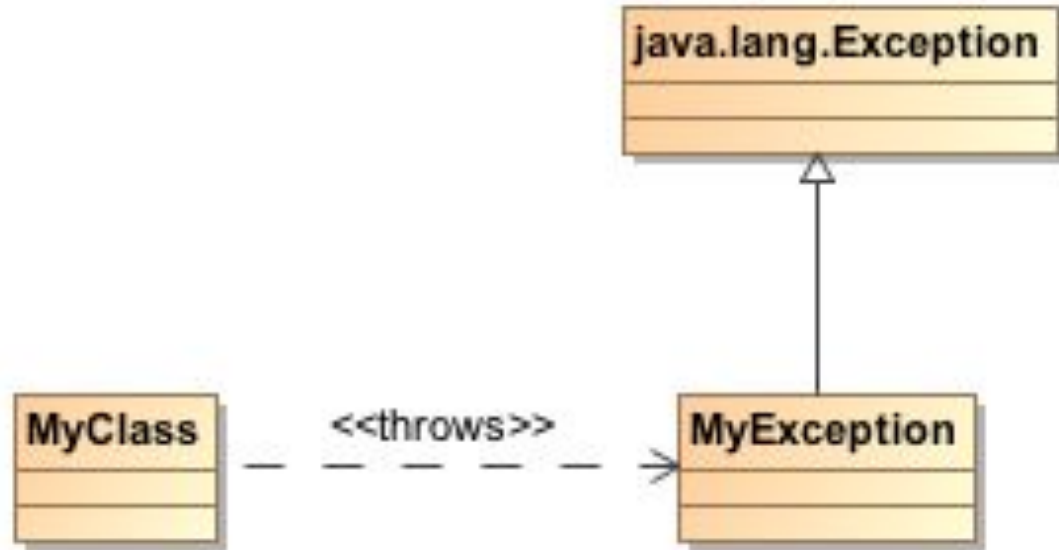
A bank account can belong to one or two customers (in case of joint account)

Final Multiplicity

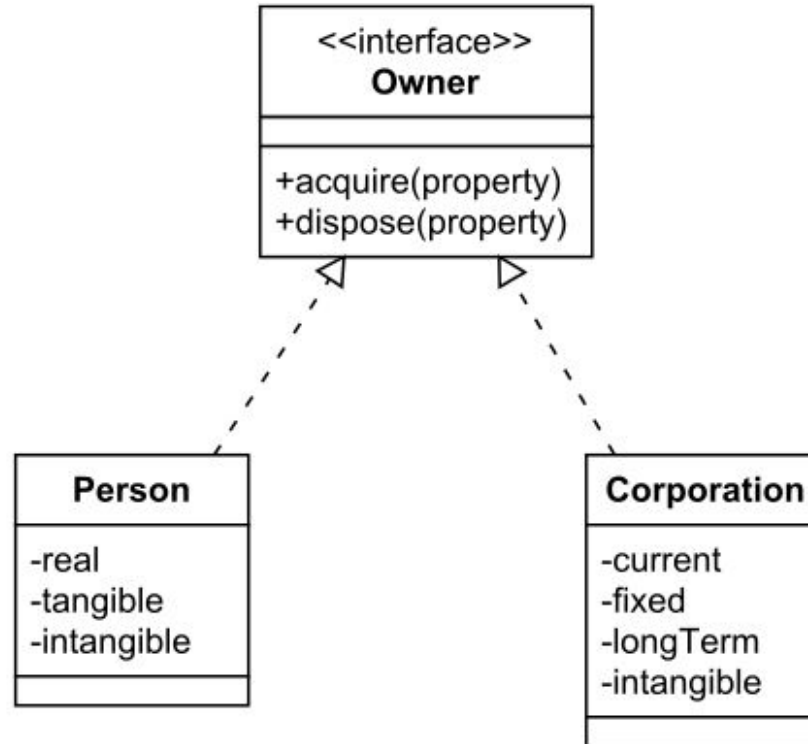


Both multiplicities combined in a single diagram

Exceptions

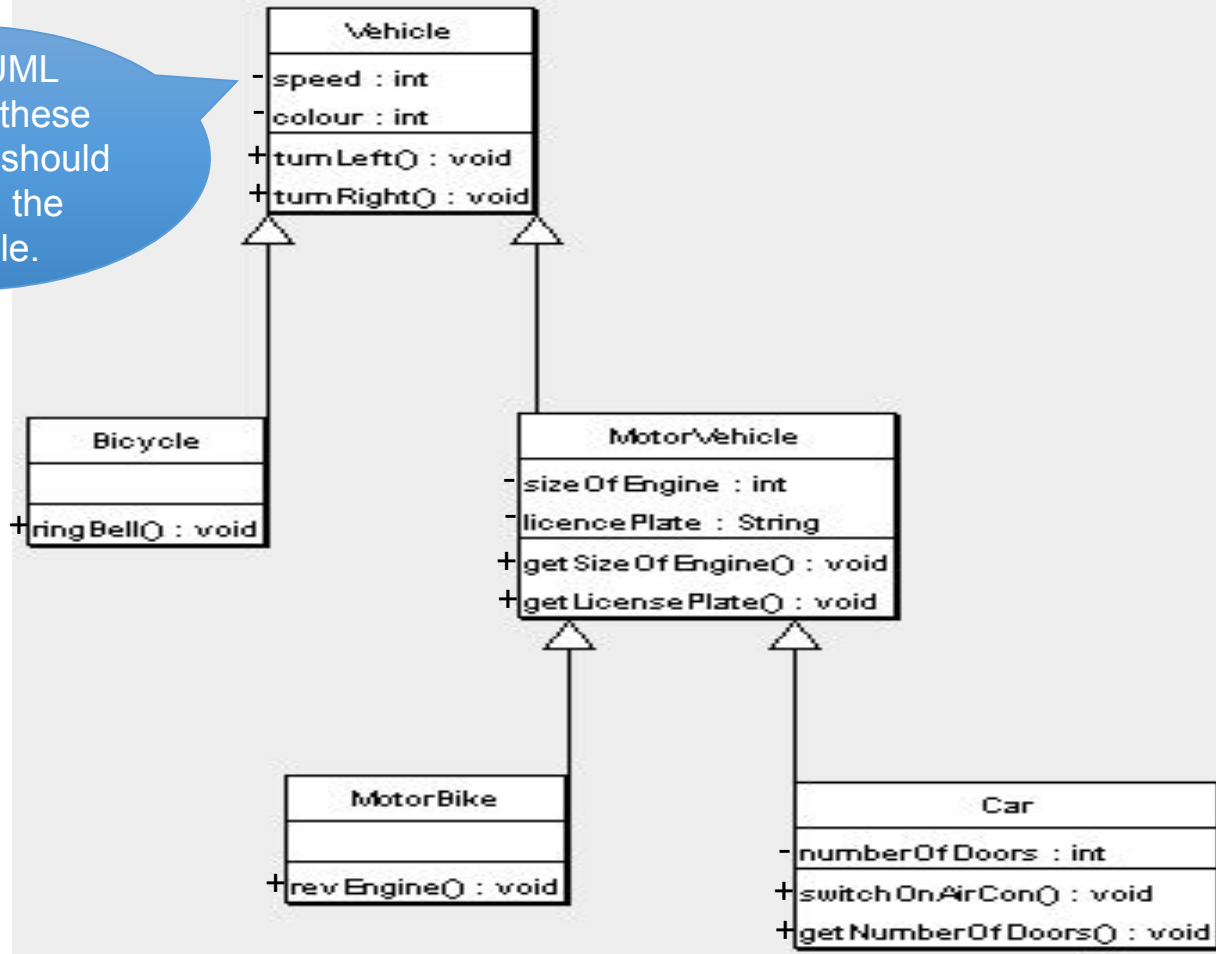


Interfaces

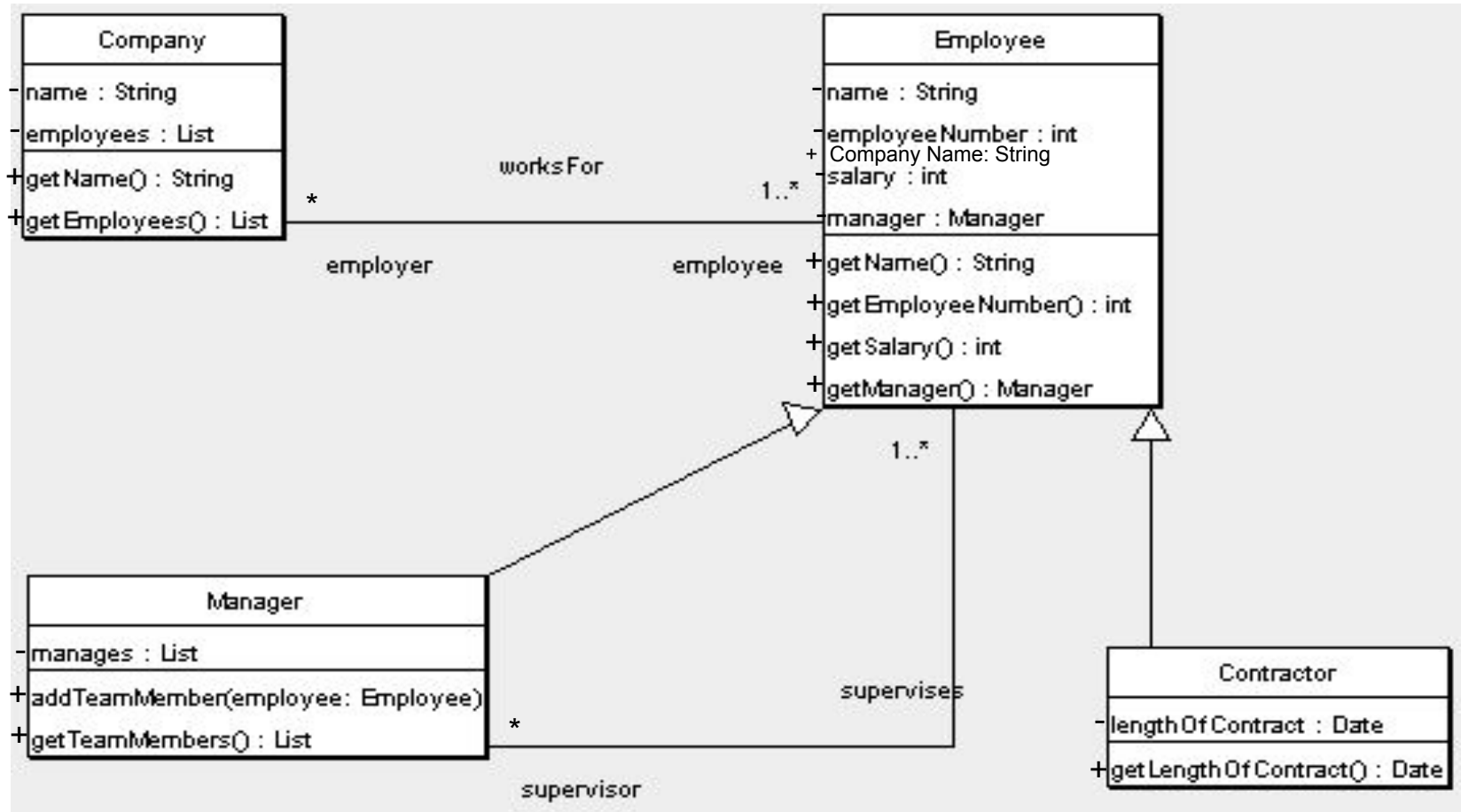


Sample Class Diagram (1/2)

In your UML diagrams, these “+”, “-”, etc, should be inside the rectangle.



Sample Class Diagram (2/2)

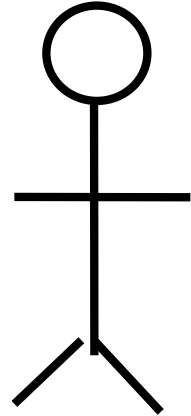


UML Diagrams: Use Cases

- Means of capturing requirements
 - Used at a very early phase of software development for requirement gathering (analysis phase)
 - Provides a high level overview of the system
 - Class diagrams are created after generating use case diagrams
- Document interactions between user(s) and the system
 - User (actor) is not part of the system itself
 - But an actor can be *another* system
- **Use case diagrams** describe what a system does from the standpoint of an external observer. The emphasis is on *what* a system does rather than *how*

Actors in Use Case

- What is an Actor?
 - A user or outside system that interacts with the system being designed in order to obtain some value from that interaction
 - It can be a:
 - Human
 - Peripheral device (hardware)
 - External system or subsystem
 - Time or time-based event
 - Labelled using a descriptive noun or phrase
 - Represented by stick figure



Use Case Analysis (1/4)

- Sample scenario
 - *“A patient calls the clinic to make an appointment for a yearly checkup. The receptionist finds the nearest empty time slot in the appointment book and schedules the appointment for that time slot”*
- We want to write a use case for this scenario

Use Case Analysis (2/4)

- Sample scenario
 - *“A patient calls the clinic to make an appointment for a yearly checkup. The receptionist finds the nearest empty time slot in the appointment book and schedules the appointment for that time slot”*
- Who is the actor?
 - The actor is a “Patient” here

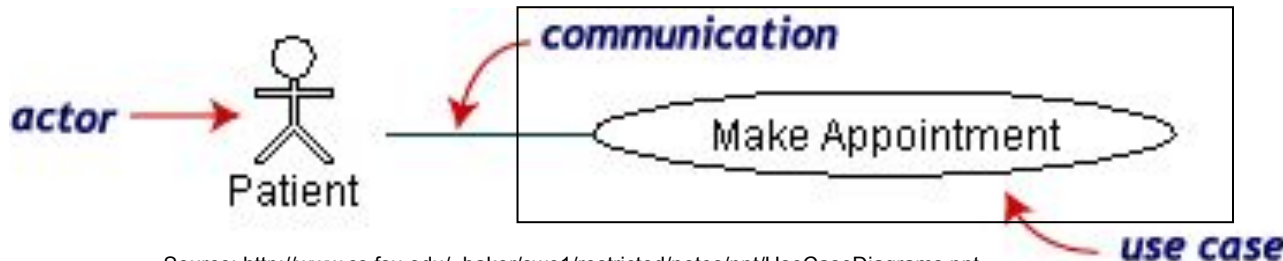


Use Case Analysis (3/4)

- Sample scenario
 - *“A patient calls the clinic to make an appointment for a yearly checkup. The receptionist finds the nearest empty time slot in the appointment book and schedules the appointment for that time slot”*
- A use case describes a function that a system performs to achieve the user’s goal.
- A use case must yield an observable result that is of value to the user of the system.
 - So, what is the use case here?
 - The use case is “Make Appointment”

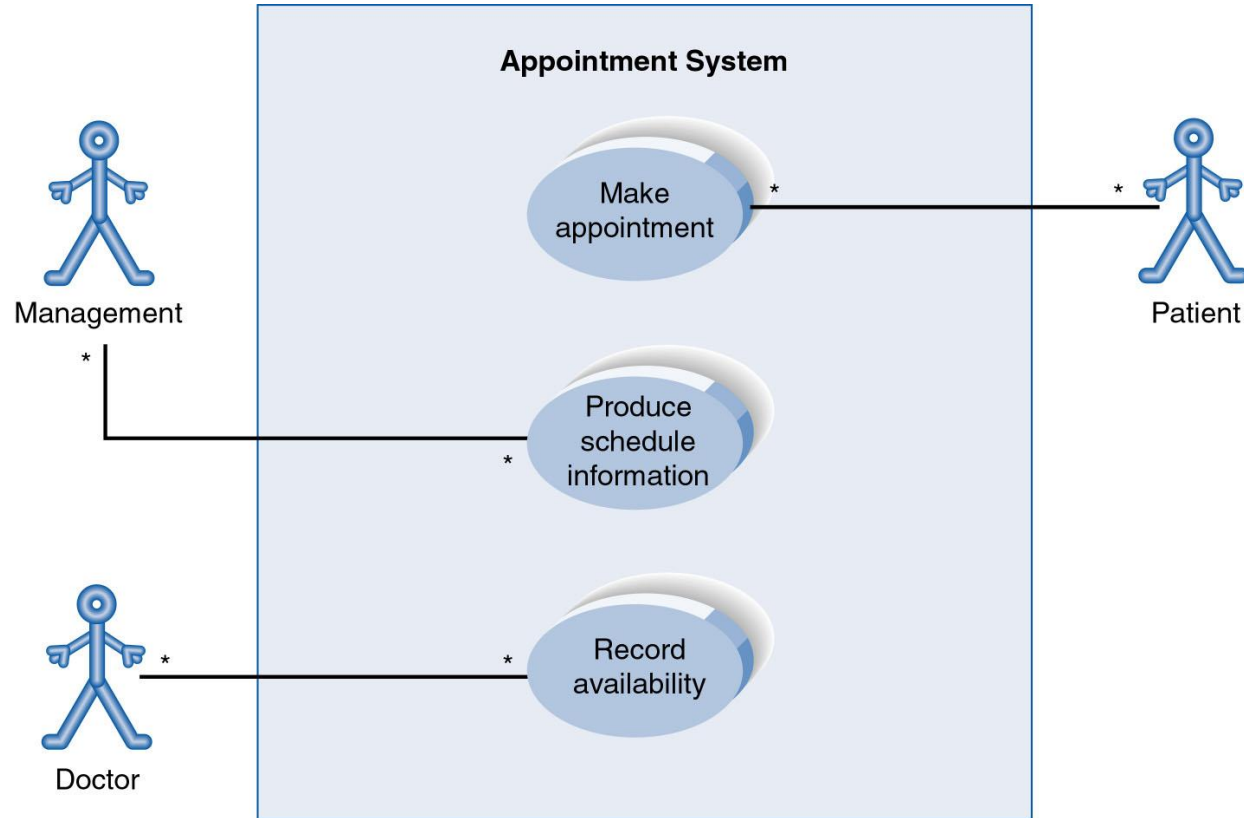
Use Case Analysis (4/4)

- The picture below is a **Make Appointment** use case for the medical clinic.
- The actor is a **Patient**. The connection between actor and use case is a **communication**
- Actors are stick figures
- Use cases are ovals
 - Labelled using a descriptive verb-noun phrase
- Communications are lines that link actors to use cases
- Boundary rectangle is placed around the perimeter of the system to show how the actors communicate with the system



Use Case Diagram

- A use case diagram is a collection of actors, use cases, and their communications

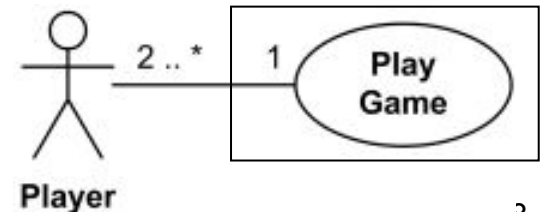
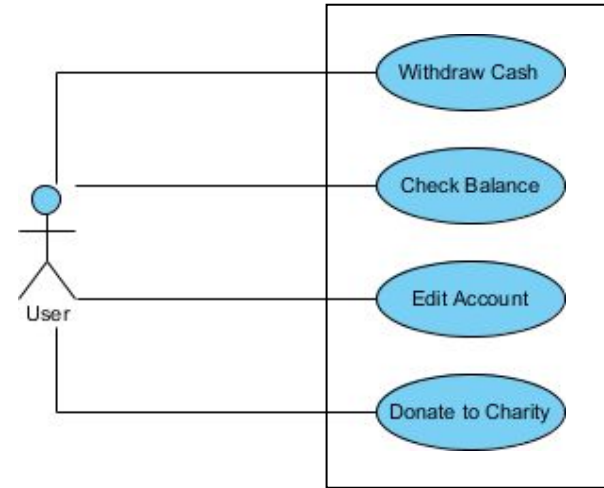


Relationships for Use Cases

- Association
- Generalization
- Extend
- Include

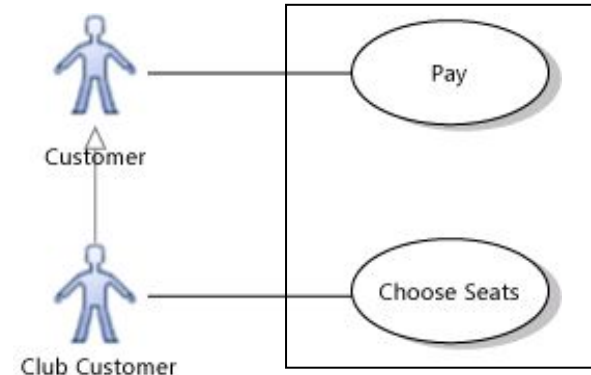
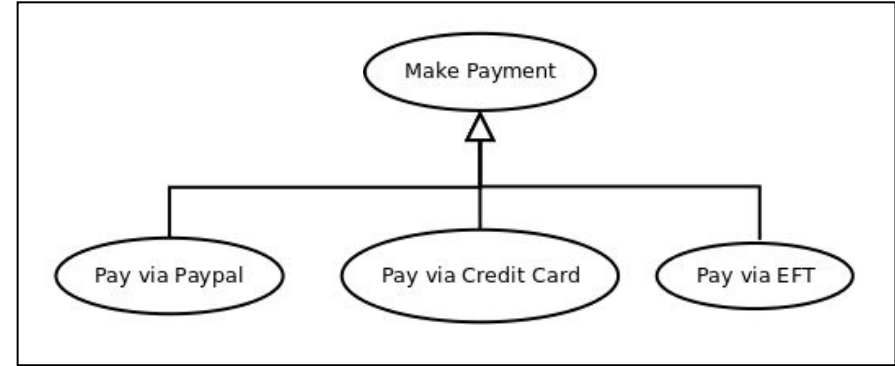
Association Relationship

- Exists only between an actor and a use case
 - Indicates that an actor can use certain functionality of the system
- Represented by a solid line without arrowhead
 - Most commonly used representation
 - Uncommon to show one-way association
- The association between an actor and a use case can also show multiplicity at each end



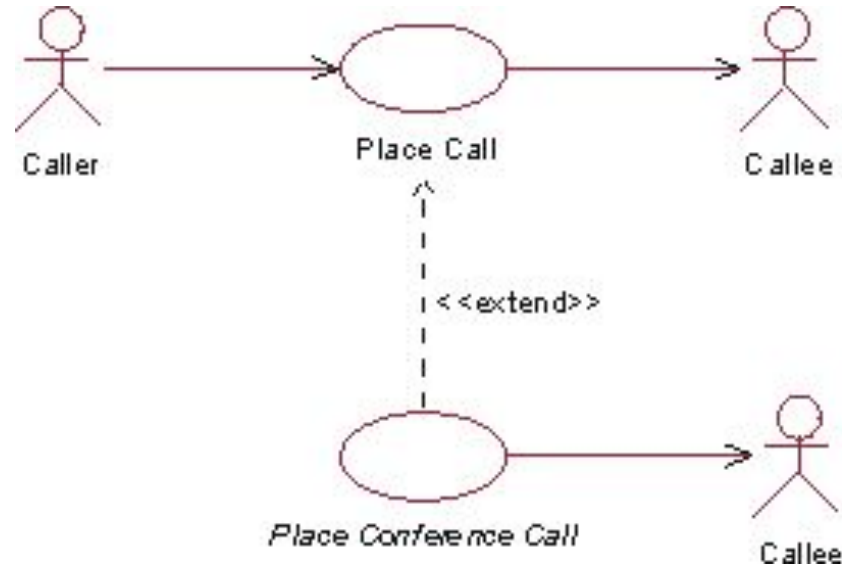
Generalization Relationship

- Could exist between two actors or between two use cases
 - Indicates parent/child relationship
- Represented by a solid line with a triangular and hollow arrowhead
 - From child to parent



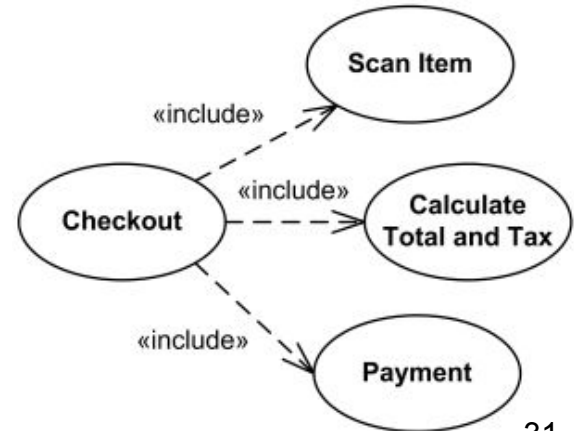
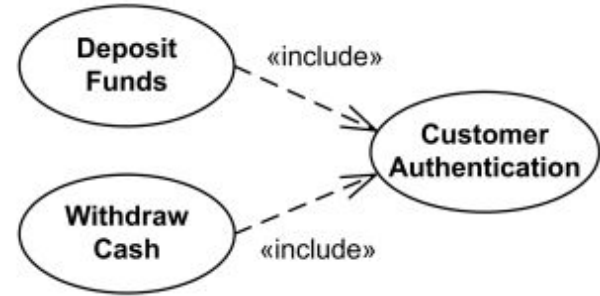
Extend Relationship “<<extend>>”

- Exists only between use cases
 - This relationships represent optional or seldom invoked cases
 - Indicates that although one use case is a variation of another but it is invoked rarely
 - Lot of shared code between these use cases (**not to be confused with inheritance**)
- Represented using a dashed arrow with an arrowhead. The notation “<< extend >>” is also mentioned above the arrow
 - The direction of the arrow is toward the extended use cases

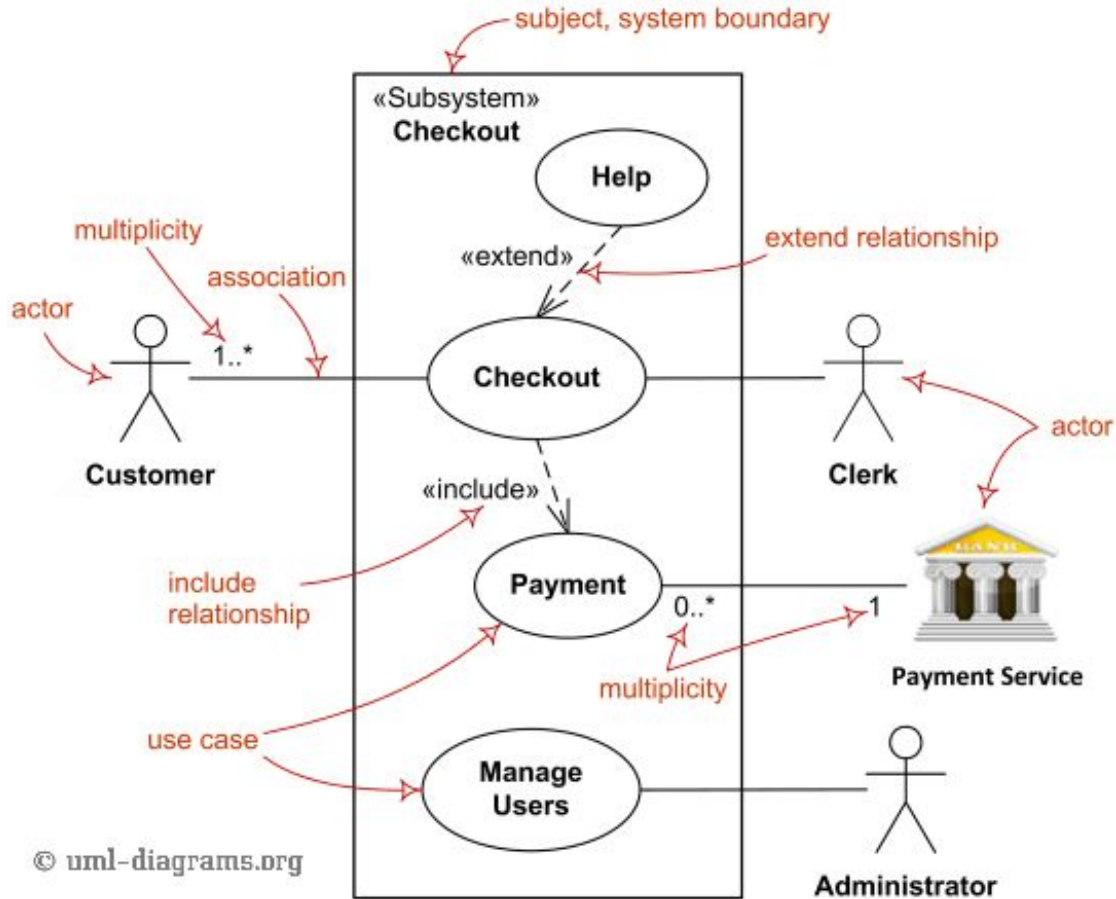


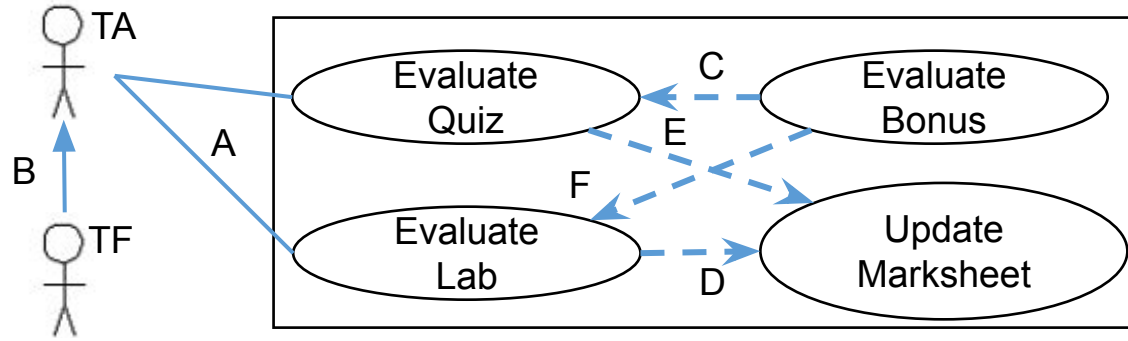
Include Relationship “<<include>”

- Exists only between use cases
 - Represents behavior that is factored out of the use case
 - Doesn't mean that the factored out use case is an optional or seldom invoked cases
- Represented using a dashed arrow with an arrowhead. The notation “<< include>>” is also mentioned above the arrow
 - The direction of the arrow is toward the included use case



Sample Use Case





Sequence Diagrams

A sequence diagram simply depicts interaction between objects in a sequential order i.e. the order in which these interactions take place.

